

VERILOG IMPLEMENTATION OF A **CACHE-COHERENT MULTIPROCESSOR**

Roshan Sundaram

BACKGROUND

Each class has taught us something **new**:

- *ECE 250*: basic single-cycle core in Logisim
- *ECE 350*: pipelined core in Verilog
- *ECE 552*: high-performance core in gem5
- ***ECE 652*: shared-memory multiprocessor**

MOTIVATION

Building hardware **is fun!**

Why build hardware?

- 1 Learned about coherence $\implies$ build coherence protocol
- 2 Understand real-world behavior
- 3 Practice Verilog and debugging

Focus on shared memory interface and protocol FSM

GOALS

Make it work

Doesn't need to be fast...needs to work

Implement modern ISA

MIPS was fun, but RISC-V is modern and popular

2 Cores > 1 Core?

How hard is multithreaded programming? Let's find out...

COHERENCE PROTOCOL

MSI Snooping Protocol implemented with a shared bus

- Independent L1I caches preloaded with instructions
- Miss in L1Ds & Access L2: 6 cycles
- Get data from other core: 4 cycles
- Hit in local L1D: 1 cycle

Consistency: RISC-V follows release consistency, but we implement sequential consistency

HARDWARE OVERVIEW

- *Autogenerated* WARP-V single-cycle core
- Simple cache designs (write-allocate, write-back)
- Dual-ported L1D for snooping

Bus Arbiter

- CORE 0, CORE 1, L2
- Round-Robin priority
- Snooping signals
- Concurrent requests

L1D Cache Controller

- MSI state machine
- Handles dataflow
- Core to core link
- CPU stall logic

Microbenchmarks:

- Independent, simultaneous core operation
- Basic sharing: CORE 0 writes $\Rightarrow$ CORE 1 reads
- Cores obey coherence protocol on repeated writes/reads
- Fast-paced false-sharing
- Cache block evicted and fetched from L2
- CORE 0 populates array, CORE 1 calculates sum

Use GTKWave debugging tool to trace register values, bus arbiter actions, and coherence messages

NEXT STEPS

Lots of **room for improvement!**

This weekend:

- Better workloads and benchmarks (SPLASH-2?)
- Register file inspector and autotester
- Synthesize in Vivado

Long-term:

- Directory coherence protocol
- Parameterization for scalability
- Higher-performance cores
- Improve programmability